NAVIGATING THE FUTURE OF ONLINE SHOPPING

E-COMMERCE DATA ANALYSIS USING SQL & PYTHON

BY- GOURI SHARMA

ABOUT COMPANY

The dataset represents a multi-category e-commerce platform connecting customers and sellers across various regions. The platform facilitates online transactions, offering a wide range of products with multiple payment options including installments.

With increasing competition in the e-commerce space, understanding customer behavior, sales performance, and seller contribution is crucial for business growth

TOOLS

- SQL (MySQL) → Data extraction & querying
- Python (Pandas, Matplotlib) → Data analysis & visualization
- Google Colab → Implementation
- Structured workflow: Data Cleaning → Analysis → Visualization → Insights

Q1. UNIQUE CITIES WHWRE CUSTOMERS ARE LOCATED

PYTHON CODE

```
customers['customer_city'].unique()
```

SQL QUEREY

```
SELECT DISTINCT customer_city
FROM customers;
```

OUTPUT

```
array(['franca', 'sao bernardo do campo', 'sao paulo', ...,
       'monte bonito', 'sao rafael', 'eugenio de castro'], dtype=object)
```

OUTPUT

customer_city
franca
sao bernardo do campo
sao paulo
mogi das cruzes
campinas
jaragua do sul
timoteo
curitiba

Q2. COUNT OF NO. OF ORDERS PLACED IN 2017

PYTHON CODE

```python
orders['order_purchase_timestamp'] = pd.to_datetime(orders['order_purchase_timestamp'])

orders_2017 = orders[orders['order_purchase_timestamp'].dt.year == 2017]['order_id'].nunique()

orders_2017
```

OUTPUT

```
45101
```

SQL QUEREY

```sql
SELECT COUNT(*) AS total_orders_2017
FROM orders
WHERE YEAR(order_purchase_timestamp) = 2017;
```

OUTPUT

	total_orders_2017
▶	45101

Q3. TOTAL SALES PER CATEGORY

PYTHON CODE

```python
temp = order_items.merge(
    products[['product_id', 'product_category']],
    on='product_id',
    how='inner'
)
temp['product_category'] = temp['product_category'].fillna('Unknown')
category_sales = temp.groupby('product_category')['price'].sum().reset_index()
category_sales.rename(columns={
    'product_category': 'product_category_name',
    'price': 'total_sales'
}, inplace=True)

category_sales = category_sales.sort_values(
    by=['total_sales', 'product_category_name'],
    ascending=[False, True]
)
top9 = category_sales.head(9)
top9 = top9.reset_index(drop=True)
top9
```

SQL QUEREY

```sql
SELECT p.product_category_name,
       ROUND(SUM(oi.price),2) AS total_sales
FROM order_items oi
JOIN products p
ON oi.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY total_sales DESC;
```

OUTPUT

product_category_name	total_sales
HEALTH BEAUTY	1258681.34
Watches present	1205005.68
bed table bath	1036988.68
sport leisure	988048.97
computer accessories	911954.32
Furniture Decoration	729762.49
Cool Stuff	635290.85
housewares	632248.66
automotive	592720.11

OUTPUT

HEALTH BEAUTY	1258681.34
Watches present	1205005.68
bed table bath	1036988.68
sport leisure	988048.97
computer accessories	911954.32
Furniture Decoration	729762.49
Cool Stuff	635290.85

Q4. % OF ORDERS PLACED IN INSTALLMENTS

PYTHON CODE

```python
# ------------------------------
# STEP 1: Total unique orders
# ------------------------------
total_orders = payments['order_id'].nunique()

# ------------------------------
# STEP 2: Orders with installments > 1
# ------------------------------
installment_orders = payments[payments['payment_installments'] > 1]['order_id'].nunique()

# ------------------------------
# STEP 3: Percentage calculation
# ------------------------------
percentage = (installment_orders / total_orders) * 100

# ------------------------------
# STEP 4: Round to match SQL
# ------------------------------
percentage = round(percentage, 2)

percentage
```

SQL QUEREY

```sql
SELECT

ROUND(

    COUNT(DISTINCT CASE WHEN payment_installments > 1 THEN order_id END) * 100.0

    / COUNT(DISTINCT order_id), 2

) AS installment_percentage

FROM payments;
```

OUTPUT

```
51.46
```

OUTPUT

installment_percentage
51.46

Q5. NO. OF CUSTOMERS FROM EACH STATE

PYTHON CODE

```python
customers_state = customers['customer_state'].value_counts()

customers_state
```

SQL QUEREY

```sql
SELECT customer_state,
       COUNT(*) AS total_customers
FROM customers
GROUP BY customer_state
ORDER BY total_customers DESC;
```

OUTPUT

customer_state	
SP	41746
RJ	12852
MG	11635
RS	5466
PR	5045
SC	3637
BA	3380

OUTPUT

customer_state	total_customers
SP	41745
RJ	12852
MG	11635
RS	5466
PR	5045
SC	3637

Q6. NO. OF ORDERS PLACED IN 2018

PYTHON CODE

```python
orders['order_purchase_timestamp'] = pd.to_datetime(orders['order_purchase_timestamp'])

orders['year_month'] = orders['order_purchase_timestamp'].dt.to_period('M').astype(str)

orders_2018 = orders[orders['order_purchase_timestamp'].dt.year == 2018]

orders_count = orders_2018.groupby('year_month')['order_id'].count().reset_index()
orders_count.rename(columns={'order_id': 'no_of_orders'}, inplace=True)

orders_count = orders_count.sort_values('year_month')

orders_count = orders_count.reset_index(drop=True)

orders_count
```

SQL QUEREY

```sql
SELECT
    DATE_FORMAT(order_purchase_timestamp, '%Y-%m') AS month,
    COUNT(*) AS total_orders
FROM orders
WHERE YEAR(order_purchase_timestamp) = 2018
GROUP BY month
ORDER BY month;
```

OUTPUT

year_month	no_of_orders
2018-01	7269
2018-02	6728
2018-03	7211
2018-04	6939
2018-05	6873
2018-06	6167
2018-07	6292
2018-08	6512

OUTPUT

month	total_orders
2018-01	7269
2018-02	6728
2018-03	7211
2018-04	6939
2018-05	6873
2018-06	6167
2018-07	6292
2018-08	6512

Q7. AVG NO. OF PRODUCTS PER ORDER BY CUSTOMER CITY

PYTHON CODE

```python
products_per_order = order_items.groupby('order_id').size().reset_index(name='product_count')
merged = products_per_order.merge(
    orders[['order_id', 'customer_id']],
    on='order_id'
).merge(
    customers[['customer_id', 'customer_city']],
    on='customer_id'
)

result = merged.groupby('customer_city')['product_count'].mean().reset_index()


result['avg_products_per_order'] = result['product_count'].round(2)

result = result[['customer_city', 'avg_products_per_order']]
result = result.sort_values(
    by=['avg_products_per_order', 'customer_city'],
    ascending=[False, True]
)
result.head(10)
```

SQL QUEREY

```sql
SELECT
    c.customer_city,
    ROUND(AVG(t.product_count), 2) AS avg_products_per_order
FROM (
    SELECT
        oi.order_id,
        COUNT(*) AS product_count
    FROM order_items oi
    GROUP BY oi.order_id
) t
JOIN orders o ON t.order_id = o.order_id
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_city
ORDER BY avg_products_per_order DESC;
```

OUTPUT

customer_city	avg_products_per_order
padre carvalho	7.0
celso ramos	6.5
candido godoi	6.0
datas	6.0
matias olimpio	5.0
cidelandia	4.0

OUTPUT

customer_city	avg_products_per_order
padre carvalho	7.00
celso ramos	6.50
candido godoi	6.00
datas	6.00
matias olimpio	5.00
cidelandia	4.00
picarra	4.00
curralinho	4.00

Q8. REVENUE CONTRIBUTION OF EACH CATEGORY

PYTHON CODE

```python
temp = order_items.merge(
    products[['product_id', 'product_category']],
    on='product_id',
    how='inner'
)
category_revenue = temp.groupby('product_category')['price'].sum().reset_index()
category_revenue.rename(columns={
    'product_category': 'product_category_name',
    'price': 'revenue_category'
}, inplace=True)
total_revenue = category_revenue['revenue_category'].sum()


category_revenue['revenue_percentage'] = (
    category_revenue['revenue_category'] / total_revenue
) * 100

category_revenue['revenue_category'] = category_revenue['revenue_category'].round(6)
category_revenue['revenue_percentage'] = category_revenue['revenue_percentage'].round(2)

category_revenue = category_revenue.sort_values(
    by=['revenue_category', 'product_category_name'],
    ascending=[False, True]
)
category_revenue = category_revenue.reset_index(drop=True)
category_revenue
```

SQL QUEREY

```sql
SELECT * FROM products LIMIT 1;

SELECT
    p.product_category_name,
    SUM(oi.price) AS category_revenue,
    ROUND(
        SUM(oi.price) * 100.0 /
        (SELECT SUM(price) FROM order_items),
        2
    ) AS revenue_percentage
FROM order_items oi
JOIN products p
    ON oi.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY category_revenue DESC;
```

OUTPUT

product_category_name	revenue_category	revenue_percentage
HEALTH BEAUTY	1258681.34	9.26
Watches present	1205005.68	8.87
bed table bath	1036988.68	7.63
sport leisure	988048.97	7.27
computer accessories	911954.32	6.71

OUTPUT

product_category_name	category_revenue	revenue_percentage
HEALTH BEAUTY	1258681.340969324	9.26
Watches present	1205005.6775493622	8.87
bed table bath	1036988.6800355911	7.63
sport leisure	988048.9688892365	7.27
computer accessories	911954.3174581528	6.71
Furniture Decoration	729762.4922962189	5.37
Cool Stuff	635290.8516969681	4.67
housewares	632248.6608705521	4.65

Q9. CORELATION : PRODUCT PRICE VS PURCHASE FREQUENCY

PYTHON CODE

```python
# Frequency
product_freq = order_items.groupby('product_id').size().reset_index(name='purchase_count')

# Avg price
product_price = order_items.groupby('product_id')['price'].mean().reset_index()

# Merge
merged = product_freq.merge(product_price, on='product_id')

# Correlation
correlation = merged['price'].corr(merged['purchase_count'])

correlation
```

SQL QUEREY

```sql
SELECT
(
    COUNT(*) * SUM(avg_price * purchase_count)
    - SUM(avg_price) * SUM(purchase_count)
)
/
SQRT(
    (COUNT(*) * SUM(avg_price * avg_price) - POWER(SUM(avg_price), 2)) *
    (COUNT(*) * SUM(purchase_count * purchase_count) - POWER(SUM(purchase_count), 2))
) AS correlation
FROM (
    SELECT
        product_id,
        COUNT(*) AS purchase_count,
        AVG(price) AS avg_price
    FROM order_items
    GROUP BY product_id
) t;
```

OUTPUT

```
np.float64(-0.032139862680945167)
```

OUTPUT

	correlation
▶	-0.032139862547781 45

Q10. REVENUE BY SELLER AND RANKING

PYTHON CODE

```python
seller_revenue = order_items.groupby('seller_id')['price'].sum().reset_index()

seller_revenue.rename(columns={'price': 'revenue'}, inplace=True)

seller_revenue['revenue'] = seller_revenue['revenue'].round(6)

seller_revenue = seller_revenue.sort_values(
    by=['revenue', 'seller_id'],
    ascending=[False, True]
)

seller_revenue['rnk'] = seller_revenue['revenue'].rank(
    method='dense',
    ascending=False
).astype(int)

seller_revenue = seller_revenue.reset_index(drop=True)

seller_revenue.head(10)
```

SQL QUEREY

```sql
SELECT
    oi.seller_id,
    SUM(oi.price) AS revenue,
    RANK() OVER (ORDER BY SUM(oi.price) DESC) AS rnk
FROM order_items oi
GROUP BY oi.seller_id;
```

OUTPUT

seller_id	revenue	rnk
4869f7a5dfa277a7dca6462dcf3b52b2	229472.63	1
53243585a1d6dc2643021fd1853d8905	222776.05	2
4a3ca9315b744ce9f8e9374361493884	200472.92	3
fa1c13f2614d7b5c4749cbc52fecda94	194042.03	4
7c67e1448b00f6e969d365cea6b010ab	187923.89	5
7e93a43ef30c4f03f38b393420bc753a	176431.87	6
da8622b14eb17ae2831f4ac5b9dab84a	160236.57	7
7a67c85e85bb2ce8582c35f2203ad736	141745.53	8
1025f0e2d44d7041d6cf58b6550e0bfa	138968.55	9
955fee9216a65b617aa5c0531780ce60	135171.70	10

OUTPUT

seller_id	revenue	rnk
4869f7a5dfa277a7dca6462dcf3b52b2	229472.6283493042	1
53243585a1d6dc2643021fd1853d8905	222776.0495452881	2
4a3ca9315b744ce9f8e9374361493884	200472.921459198	3
fa1c13f2614d7b5c4749cbc52fecda94	194042.02939605713	4
7c67e1448b00f6e969d365cea6b010ab	187923.8919391632	5
7e93a43ef30c4f03f38b393420bc753a	176431.86933135986	6
da8622b14eb17ae2831f4ac5b9dab84a	160236.5680885315	7
7a67c85e85bb2ce8582c35f2203ad736	141745.53166007996	8

Q11.MOVING AVERAGE OF ORDER VALUES FOR EACH CUSTOMER OVER THEIR ORDER HISTORY

PYTHON CODE

```python
orders['order_purchase_timestamp'] = pd.to_datetime(orders['order_purchase_timestamp'])

order_value = order_items.groupby('order_id')['price'].sum().reset_index()
order_value.rename(columns={'price': 'order_value'}, inplace=True)

order_value = order_value.merge(
    orders[['order_id', 'customer_id', 'order_purchase_timestamp']],
    on='order_id',
    how='inner'
)

order_value = order_value.sort_values(['customer_id', 'order_purchase_timestamp'])


order_value['moving_avg'] = order_value.groupby('customer_id')['order_value'] \
    .transform(lambda x: x.rolling(window=3, min_periods=1).mean())

order_value = order_value[[
    'customer_id',
    'order_id',
    'order_purchase_timestamp',
    'order_value',
    'moving_avg'
]]

order_value = order_value.reset_index(drop=True)
order_value.head(10)
```

SQL QUEREY

```sql
FROM order_value
ORDER BY customer_id, order_purchase_timestamp;
 WITH order_value AS (
    SELECT
        c.customer_unique_id,
        o.order_id,
        o.order_purchase_timestamp,
        SUM(oi.price) AS order_value
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    JOIN customers c ON o.customer_id = c.customer_id
    GROUP BY c.customer_unique_id, o.order_id, o.order_purchase_timestamp
)
SELECT
    customer_unique_id,
    order_id,
    order_purchase_timestamp,
    order_value,
AVG(order_value) OVER (
        PARTITION BY customer_unique_id
        ORDER BY order_purchase_timestamp
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS moving_avg

FROM order_value
ORDER BY customer_unique_id, order_purchase_timestamp;
```

OUTPUT

customer_id	order_id	order_purchase_timestamp	order_value	moving_avg
00012a2ce6f8dcda20d059ce98491703	5f79b5b0931d63f1a42989eb65b9da6e	2017-11-14 16:08:26	89.80	89.80
000161a058600d5901f007fab4c27140	a44895d095d7e0702b6a162fa2dbeced	2017-07-16 09:40:32	54.90	54.90
0001fd6190edaaf884bcaf3d49edf079	316a104623542e4d75189bb372bc5f8d	2017-02-28 11:06:43	179.99	179.99
0002414f95344307404f0ace7a26f1d5	5825ce2e88d5346438686b0bba99e5ee	2017-08-16 13:09:20	149.90	149.90
000379cdec625522490c315e70c7a9fb	0ab7fb08086d4af9141453c91878ed7a	2018-04-02 13:42:17	93.00	93.00

OUTPUT

customer_id	order_id	order_purchase_timestamp	order_value	moving_avg
00012a2ce6f8dcda20d059ce98491703	5f79b5b0931d63f1a42989eb65b9da6e	2017-11-14 16:08:26	89.80000305175781	89.80000305175781
000161a058600d5901f007fab4c27140	a44895d095d7e0702b6a162fa2dbeced	2017-07-16 09:40:32	54.900001525878906	54.900001525878906
0001fd6190edaaf884bcaf3d49edf079	316a104623542e4d75189bb372bc5f8d	2017-02-28 11:06:43	179.9900049316406	179.9900049316406
0002414f95344307404f0ace7a26f1d5	5825ce2e88d5346438686b0bba99e5ee	2017-08-16 13:09:20	149.8999389648438	149.8999389648438
000379cdec625522490c315e70c7a9fb	0ab7fb08086d4af9141453c91878ed7a	2018-04-02 13:42:17	93	93

Q12. COMULATIVE SALES PER MONTH OF EACH YEAR

PYTHON CODE

```python
temp = orders.merge(order_items, on='order_id', how='inner')

temp['order_purchase_timestamp'] = pd.to_datetime(temp['order_purchase_timestamp'])

temp['year'] = temp['order_purchase_timestamp'].dt.year
temp['month'] = temp['order_purchase_timestamp'].dt.month
temp['month_name'] = temp['order_purchase_timestamp'].dt.month_name(
monthly_sales = temp.groupby(['year', 'month', 'month_name'])['price'].sum().reset_index
monthly_sales = monthly_sales.sort_values(['year', 'month'])

monthly_sales['cumulative_sales'] = monthly_sales.groupby('year')['price'].cumsum()

monthly_sales.rename(columns={'price': 'monthly_sales'}, inplace=True)

monthly_sales = monthly_sales[
    ['year', 'month', 'month_name', 'monthly_sales', 'cumulative_sales']
]
monthly_sales = monthly_sales.reset_index(drop=True)

monthly_sales['monthly_sales'] = monthly_sales['monthly_sales'].round(6)
monthly_sales['cumulative_sales'] = monthly_sales['cumulative_sales'].round(6
monthly_sales.head(10)
```

SQL QUEREY

```sql
SELECT
    YEAR(o.order_purchase_timestamp) AS year,
    MONTH(o.order_purchase_timestamp) AS month,
    MONTHNAME(o.order_purchase_timestamp) AS month_name,

    SUM(oi.price) AS monthly_sales,

    SUM(SUM(oi.price)) OVER (
        PARTITION BY YEAR(o.order_purchase_timestamp)
        ORDER BY MONTH(o.order_purchase_timestamp)
    ) AS cumulative_sales

FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id

GROUP BY year, month, month_name
ORDER BY year, month;
```

OUTPUT

year	month	month_name	monthly_sales	cumulative_sales
2016	9	September	267.36	267.36
2016	10	October	49507.66	49775.02
2016	12	December	10.90	49785.92
2017	1	January	120312.87	120312.87
2017	2	February	247303.02	367615.89

OUTPUT

year	month	month_name	monthly_sales	cumulative_sales
016	9	September	267.3600082397461	267.3600082397461
016	10	October	49507.65977859497	49775.01978683472
016	12	December	10.899999618530273	49785.91978645325
017	1	January	120312.86980295181	120312.86980295181
017	2	February	247303.01981592178	367615.8896188736

Q13 . YEAR ON YEAR GROWTH

PYTHON CODE

```python
temp = orders.merge(order_items, on='order_id', how='inner')
temp['order_purchase_timestamp'] = pd.to_datetime(temp['order_purchase_timestamp'])
temp['year'] = temp['order_purchase_timestamp'].dt.year
yearly_sales = temp.groupby('year')['price'].sum().reset_index()
yearly_sales.rename(columns={'price': 'total_sales'}, inplace=True)

yearly_sales = yearly_sales.sort_values('year')

yearly_sales['previous_year_sales'] = yearly_sales['total_sales'].shift(1)

yearly_sales['yoy_growth_percentage'] = (
    (yearly_sales['total_sales'] - yearly_sales['previous_year_sales']) /
    yearly_sales['previous_year_sales']
) * 100

yearly_sales['total_sales'] = yearly_sales['total_sales'].round(6)
yearly_sales['previous_year_sales'] = yearly_sales['previous_year_sales'].round(6)
yearly_sales['yoy_growth_percentage'] = yearly_sales['yoy_growth_percentage'].round(2)

yearly_sales = yearly_sales.reset_index(drop=True)
yearly_sales
```

OUTPUT

year	total_sales	previous_year_sales	yoy_growth_percentage
2016	49785.92	NaN	NaN
2017	6155806.98	49785.92	12264.55
2018	7386050.80	6155806.98	19.99

SQL QUEREY

```sql
SELECT
    year,
    total_sales,

    LAG(total_sales) OVER (ORDER BY year) AS previous_year_sales,

    ROUND(
        (total_sales - LAG(total_sales) OVER (ORDER BY year))
        / LAG(total_sales) OVER (ORDER BY year) * 100,
    2) AS yoy_growth_percentage

FROM (
    SELECT
        YEAR(o.order_purchase_timestamp) AS year,
        SUM(oi.price) AS total_sales
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY year
) t
ORDER BY year;
```

OUTPUT

year	total_sales	previous_year_sales	yoy_growth_percentage
2016	49785.91978645325	NULL	NULL
2017	6155806.979506969	49785.91978645325	12264.55
2018	7386050.802426934	6155806.979506969	19.99

Q14. CUSTOMER RETENTION RATE

PYTHON CODE

```python
# Ensure datetime
orders['order_purchase_timestamp'] = pd.to_datetime(orders['order_purchase_timestamp'])

# Sort orders
orders_sorted = orders.sort_values(['customer_id','order_purchase_timestamp'])

# First purchase
first_order = orders_sorted.groupby('customer_id')['order_purchase_timestamp'].first().reset_index()

# Merge
orders_merged = orders_sorted.merge(first_order, on='customer_id', suffixes=('','_first'))

# Days difference
orders_merged['diff_days'] = (orders_merged['order_purchase_timestamp'] -
                              orders_merged['order_purchase_timestamp_first']).dt.days

# Retained customers (within 180 days)
retained_customers = orders_merged[
    (orders_merged['diff_days'] > 0) & (orders_merged['diff_days'] <= 180)
]['customer_id'].nunique()

total_customers = orders['customer_id'].nunique()

retention_rate = (retained_customers / total_customers) * 100

retention_rate
```

SQL QUEREY

```sql
WITH first_orders AS (
    SELECT
        customer_id,
        MIN(order_purchase_timestamp) AS first_order
    FROM orders
    GROUP BY customer_id
),
repeat_customers AS (
    SELECT DISTINCT o.customer_id
    FROM orders o
    JOIN first_orders f
    ON o.customer_id = f.customer_id
    WHERE o.order_purchase_timestamp > f.first_order
    AND o.order_purchase_timestamp <= DATE_ADD(f.first_order, INTERVAL 6 MONTH)
)
SELECT
    ROUND(
        COUNT(DISTINCT r.customer_id) * 100.0 /
        COUNT(DISTINCT f.customer_id), 2
    ) AS retention_rate
FROM first_orders f
LEFT JOIN repeat_customers r
ON f.customer_id = r.customer_id;
```

OUTPUT

```
...    0.0
```

OUTPUT

	retention_rate
▶	0.00

Q15. TOP 3 CUSTOMERS WHO SPENT THE MOST MONEY IN EACH YEAR.

PYTHON CODE

```python
temp = orders.merge(order_items, on='order_id', how='inner')

temp['order_purchase_timestamp'] = pd.to_datetime(temp['order_purchase_timestamp'])

temp['year'] = temp['order_purchase_timestamp'].dt.year

cust_year_spend = temp.groupby(['year', 'customer_id'])['price'].sum().reset_index()
cust_year_spend.rename(columns={'price': 'total_spent'}, inplace=True)

cust_year_spend['rnk'] = cust_year_spend.groupby('year')['total_spent'] \
    .rank(method='min', ascending=False).astype(int)
top3 = cust_year_spend[cust_year_spend['rnk'] <= 3]
top3 = top3.sort_values(['year', 'rnk', 'customer_id'])
top3['total_spent'] = top3['total_spent'].round(6)
top3 = top3.reset_index(drop=True)
top3
```

SQL QUEREY

```sql
SELECT *
FROM (
    SELECT
        YEAR(o.order_purchase_timestamp) AS year,
        o.customer_id,
        SUM(oi.price) AS total_spent,
        RANK() OVER (
            PARTITION BY YEAR(o.order_purchase_timestamp)
            ORDER BY SUM(oi.price) DESC
        ) AS rnk
    FROM orders o
    JOIN order_items oi
    ON o.order_id = oi.order_id
    GROUP BY year, o.customer_id
) t
WHERE rnk <= 3;
```

OUTPUT

year	customer_id	total_spent	rnk
2016	a9dc96b027d1252bbac0a9b72d837fc6	1399.00	1
2016	1d34ed25963d5aae4cf3d7f3a4cda173	1299.99	2
2016	4a06381959b6670756de02e07b83815f	1199.00	3

OUTPUT

year	customer_id	total_spent	rnk
2016	a9dc96b027d1252bbac0a9b72d837fc6	1399	1
2016	1d34ed25963d5aae4cf3d7f3a4cda173	1299.989990234375	2
2016	4a06381959b6670756de02e07b83815f	1199	3

THANK YOU